

AGENT 03

Fresher Deployment Agent

(FDA)

Hybrid (Rule-Based + AI)

Agent Overview: The Fresher Deployment Agent (FDA) analyses project and resource data to evaluate whether each project or account follows the target grade pyramid of **79% Juniors, 20% Mid-level, and 1% Seniors**. It flags accounts with a low junior ratio, identifies deployment opportunities for freshers, and provides skill-based recommendations to support placement and training planning. FDA is **advisory only** — it does not modify project structures or approve fresher allocation.

Hackathon Edition	ITC Infotech Hackathon 2026	Document Version	1.0
Agent ID	Agent 03	Date of Issue	April 2026
Category	Hybrid (Rule-Based + AI)	Classification	Internal Use Only
Project Type	Task 1	RM Contact (KT)	Abdul, Phanindra

This document is **Internal Use Only**. Prepared for assigned teams of ITC Infotech Hackathon 2026 only. Do not distribute beyond your team.

1 Problem Statement & Agent Overview

1.1 Current Scenario

Fresher deployment decisions at ITC Infotech are currently driven by manual processes. Reports are obtained from the TA team, then cross-checked manually with SO data. Skills are evaluated and mapped by hand. No ready-made pyramid analysis report exists — every verification requires fresh manual preparation.

- **Current Mode:** Manual **Tools:** Team-prepared reports, SO, RIS
- **Pain Points:** No ready-made report; every request requires manual preparation; skills cannot be mapped at scale; pyramid health is invisible in real time

1.2 What the Agent Must Do

1. Ingest RIS project and resource data; load the Grade Band mapping table.
2. Classify every active resource into **Junior** / **Mid-level** / **Senior** using the configurable grade-band mapping.
3. Aggregate the grade distribution **per project and per account**.
4. Compare actual ratios to the target pyramid: **79%** / **20%** / **1%**.
5. Run four analytical rules (R1–R4) to identify imbalances and opportunities.
6. Produce a **Pyramid Health Dashboard**, **Low Junior Ratio Alerts**, **Fresher Deployment Suggestions**, and **Training Suggestions**.

1.3 Scope

In Scope					Out of Scope	
✓	Grade	pyramid	analysis	per	✗	Modifying project structures
	project/account					
✓	Low junior ratio alerts				✗	Approving fresher allocations
✓	Fresher deployment opportunity identifica-				✗	Directly updating RIS or HR systems
	tion					
✓	Skill-based deployment and training sugges-				✗	Feeding raw employee data to AI/LLM
	tions					
✓	Pyramid health dashboard output				✗	Integration with TalentX / HRMS (MVP)

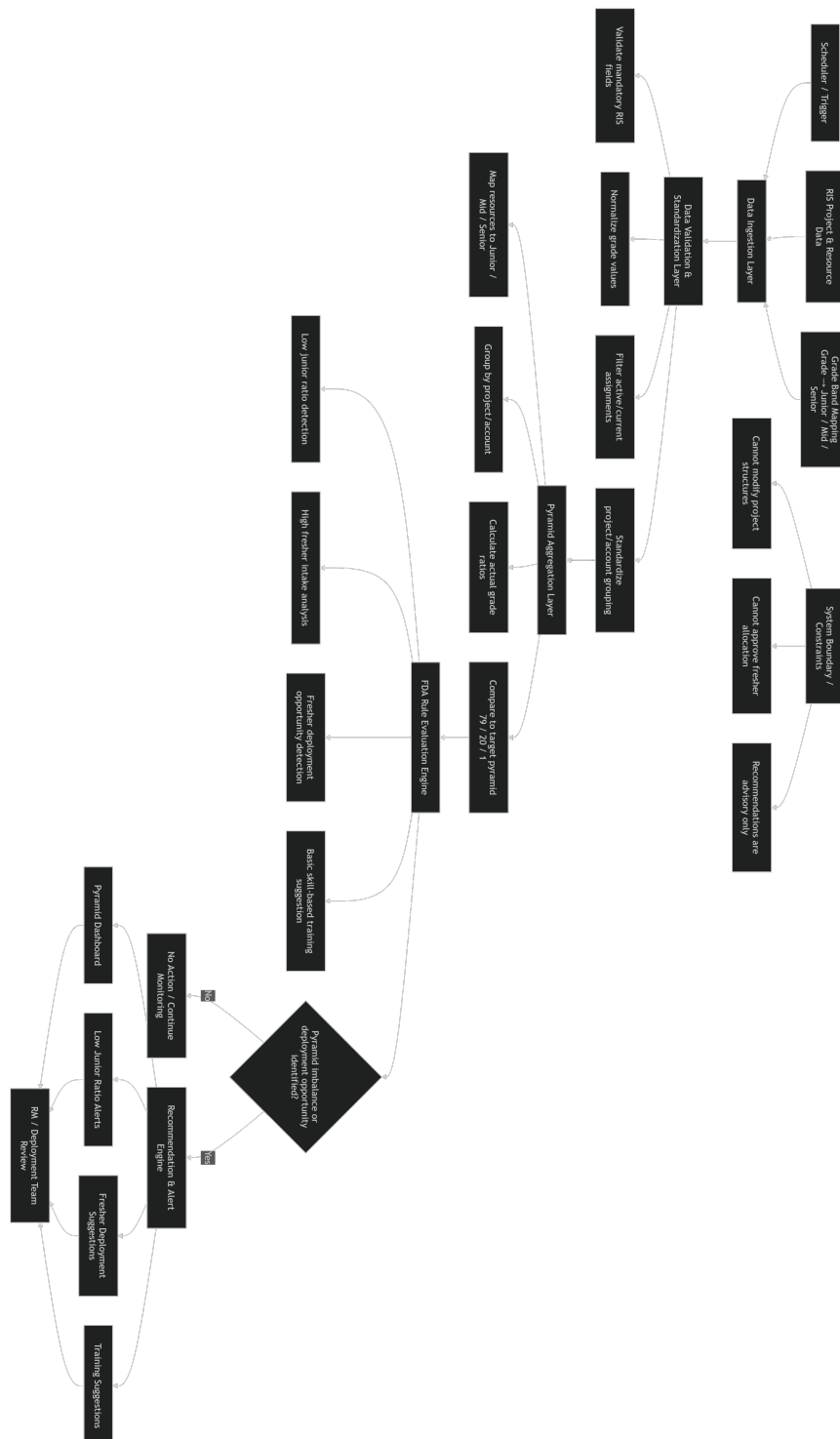
Target Pyramid

Junior	Mid-Level	Senior
79%	20%	1%

Grade-to-band mapping is maintained in a configurable external table (`grade_band_map.csv`). Bands must not be hardcoded. The authoritative mapping must be confirmed with RM contacts during the KT session.

Data Privacy Rule: Employee and project data must **NOT** be transmitted to any AI/LLM API. All analysis and scoring runs **locally only**.

2 Agent Flow Diagram



System Boundary: FDA is **advisory only**. It cannot modify project structures or approve fresher allocations. All deployment and training decisions are made by the RM / Deployment team after reviewing the FDA outputs. The next scheduled trigger revalidates the pyramid against updated RIS data.

3 Input Specification

3.1 Data Sources

The MVP uses **RIS as the sole primary source**. TalentX and HRMS integration may be added in future iterations to improve training recommendation quality. Field-level documentation is shared during the KT session with RM contacts **Abdul, Phanindra**.

Source	Role	Contains	Format
RIS Data	Primary	Project assignments, grade, skills, location, active status	Excel (.xlsx)
Grade Band Map	Reference	Configurable Grade → Junior/Mid/Senior mapping	CSV
TalentX / HRMS	Future	Richer skill profiles and fresher intake data (post-MVP)	TBD

3.2 Required RIS Fields

Priority	Fields	Used For
Must-Have	Employee ID, Employee Name, Grade , Project ID, Project Name / Account Name, Project Status, Assignment / Allocation Status, Start Date, End Date	Core pyramid calculation, active resource filter, project/account grouping
Should-Have	Project Role, Project Type / Category, Technology, Primary Skill Description, Secondary Skill Description, Location, Onsite/Offshore, RM	Deployment opportunity detection, skill-based suggestions, training themes
Good-to-Have	Experience / Experience Band, Employee Type, Department / Business Unit, Allocation Type, Personnel Status	Richer deployment scoring, population filtering, advisory output quality

3.3 Three Design Decisions to Confirm at KT

1. **Grade-to-Band Mapping:** The exact mapping of grade codes (e.g., IS1, IS2, L0, L1...) to Junior / Mid / Senior bands must be confirmed and provided as the `grade_band_map.csv` config file. Do not assume or hardcode any mapping.
2. **Unit of Analysis:** Confirm whether FDA runs at **Project ID level**, **Account level**, or both. This determines how aggregation is performed.
3. **Active Population Filter:** Confirm which records are included — active assignments only, exclude bench, exclude contractors, or all current allocations. This affects every pyramid calculation.

3.4 Input Validation Requirements

- If mandatory RIS fields are absent, halt and raise a structured error.
- Rows with null Employee ID or null Project ID must be rejected and logged.
- Grade values not present in the Grade Band Map must be logged as **unmapped grades** and excluded from pyramid calculations (not crashed).
- Date fields must be parseable; rows with unparseable dates are excluded from active-filter logic.

4 Business Rules

All four rules are mandatory. Rules R1 and R2 are **purely rule-based**. Rules R3 and R4 are **hybrid** — basic rule-based logic is the minimum requirement; more sophisticated scoring or AI-assisted reasoning earns additional evaluation points.

ID	Rule	Trigger Condition	Type	Severity
R1	Low Junior Ratio	Actual Junior % for a project/account is below 79%	Rule-Based	High
R2	High Fresher Intake	Project/account has disproportionately high junior headcount or intake velocity relative to its size	Rule-Based	Medium
R3	Fresher Deployment Opportunity	Project is junior-deficit and has roles/skills compatible with entry-level deployment	Hybrid	High
R4	Training Suggestion	Project requires specific skills; freshers mapped there need baseline training in those skills	Hybrid	Medium

Rule notes:

- **R1 — Low Junior Ratio:** Per project/account, compute: $Junior\% = \frac{JuniorHeadcount}{TotalActiveHeadcount} \times 100$. Flag if $Junior\% < 79\%$. Include the gap value ($Junior\ Gap = 79 - Junior\%$) in the output. Also flag over-indexed Mid ($>20\%$) and over-indexed Senior ($>1\%$) separately.
- **R2 — High Fresher Intake Analysis:** Identify projects/accounts where the junior count is unusually high relative to team size, or where intake rate within a recent window exceeds a configurable threshold. Exact threshold must be confirmed during KT.
- **R3 — Fresher Deployment Opportunity:** A project is a deployment candidate if: (a) it is junior-deficit per R1, and (b) at least one role on the project is consistent with entry-level deployment based on **Project Role**, **Technology**, and **Primary Skill Description**.

Scoring opportunity: A basic implementation returns a binary Yes/No flag. A stronger implementation computes a ranked **Deployment Readiness Score** per account using skill match, team size, and project type — earning additional evaluation points.

- **R4 — Training Suggestion:** For each project flagged as a deployment candidate, identify the dominant skill themes from **Technology**, **Primary Skill Description**, and **Secondary Skill Description**. Produce a suggested training theme for freshers to be deployed there.

Scoring opportunity: A basic implementation extracts the most frequent skill tag per account. A stronger implementation clusters skills semantically or uses a skill taxonomy to produce structured training recommendations — earning additional evaluation points.

No raw data may be sent to AI/LLM APIs. Any AI-assisted logic must run locally or on approved internal models.

5 Technology Stack Requirements

All submissions must use the following mandatory technology stack. Deviation from these requirements will result in disqualification.

5.1 Mandatory for All Agents

Python (Core Implementation Language)

All agent logic, rule engines, data pipelines, and utility modules must be written in **Python 3.9 or later**. Recommended libraries include **pandas**, **numpy**, **openpyxl** for data processing, **pytest** for testing, and standard logging frameworks. Python is the single language for the submission; do not mix Python with other server-side languages.

PostgreSQL (Data Storage and Persistence)

All data persistence — including configuration, intermediate results, audit logs, and output staging — must use **PostgreSQL 12 or later**. The agent must create and manage its own schema. Use **psycopg2** or **SQLAlchemy** for database access. The evaluator will provide a PostgreSQL instance; teams must document their schema creation scripts (SQL DDL) in the submission ZIP.

5.2 Mandatory for Hybrid / AI-Based Agents

▲ LangGraph / LangChain + GPT-5-Mini (For Hybrid Agents Only)

Hybrid (Rule-Based + AI) agents must use **LangGraph or LangChain** (v0.1.0 or later) as the AI orchestration framework if any AI-assisted or LLM-powered logic is implemented. This includes narrative generation, scoring models, clustering, or recommendation ranking that leverages language models.

Model Constraint: The **only permitted LLM model is GPT-5-Mini**. The provider can be any service offering GPT-5-Mini (OpenAI, Azure, or any compatible API endpoint). Teams must ensure their LangGraph/LangChain configuration explicitly specifies `model="gpt-5-mini"`. Using any other model (GPT-4, Claude, Gemini, etc.) will result in disqualification.

Important: LangGraph/LangChain must be used to manage agentic workflows and model calls — not as a data processing library. Local models (e.g., via Ollama, HuggingFace, or internal APIs) are permitted only as *substitutes* if they are explicitly named and configured to match GPT-5-Mini's capabilities. Preference: use a real GPT-5-Mini endpoint to ensure consistency across evaluations.

Note: Rule-based agents (IAI, SOVA, SRBVA) do not require LangGraph/LangChain if they contain no AI-assisted logic.

5.3 Frontend (Flexible, Not Enforced)

- **Required:** Teams are free to use any modern frontend framework — React, Vue, Angular, or plain HTML/CSS/JavaScript, or even Streamlit.
- **Backend API:** If a frontend is included, it must communicate with the Python agent via a REST or GraphQL API (Flask, FastAPI, etc.). The API must serve pre-computed agent outputs, not run live inference.
- **Evaluation:** Frontends will be evaluated on clarity and usability, not on aesthetic design. Static exports (e.g., analytics tabs as HTML) are acceptable alternatives to interactive UIs.

5.4 Dependency Management

- All Python dependencies must be declared in a `requirements.txt` file at the root of the submission.
- Database connection strings and configurations must be externalized (e.g., in a `.env` file or config file) — not hardcoded.
- The submission ZIP must include a setup script or `README.md` with clear instructions for installing dependencies, initializing the PostgreSQL schema, and configuring the LLM endpoint (if applicable).

6 Expected Output

6.1 Output Files

FDA produces two output artefacts:

- **Pyramid Analysis Report** — Excel (.xlsx), sheet FDA_PyramidReport
Naming: FDA_PyramidReport_YYYYMMDD_HHMMSS.xlsx
- **Deployment & Training Suggestions** — Excel (.xlsx), sheet FDA_Suggestions
Naming: FDA_Suggestions_YYYYMMDD_HHMMSS.xlsx

Both files use UTF-8 encoding. Use N/A for non-applicable fields.

6.2 Output Schema (Column Groups)

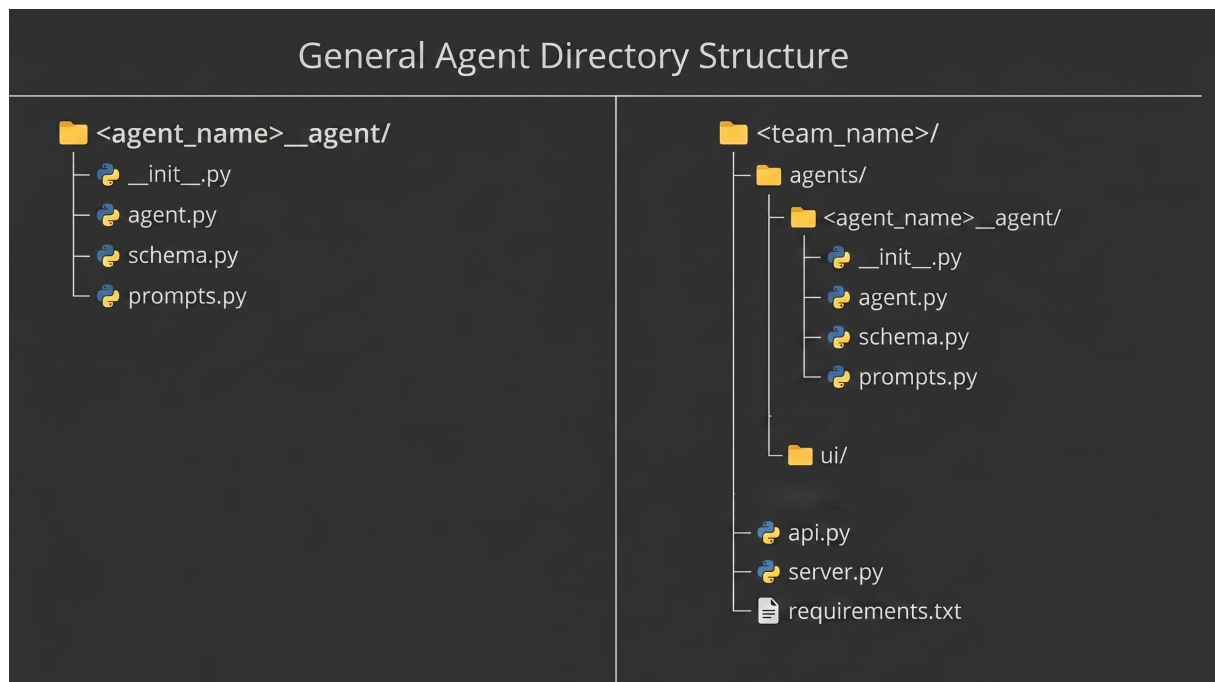
Pyramid Analysis Report — one row per project/account:

Column Group	Columns Included
Project Identity	Project ID, Project Name / Account Name, RM
Headcount	Total Active Headcount, Junior Count, Mid Count, Senior Count
Actual Ratios	Junior %, Mid %, Senior %
Target Comparison	Junior Gap (79 – Actual%), Mid Gap, Senior Gap
Pyramid Flags	Low Junior Warning, Over-Indexed Mid, Over-Indexed Senior, Pyramid Health Status
Audit	Run Date

Deployment & Training Suggestions — one row per deployment candidate account:

Column Group	Columns Included
Project Identity	Project ID, Project Name / Account Name, RM
Pyramid Summary	Junior %, Junior Gap, Pyramid Health Status
Deployment Signal	Deployment Opportunity Flag, Suggested Fresher Intake Count
Skills Context	Relevant Technologies, Primary Skills Present
Training	Suggested Training Theme, Training Suggestion Flag
Scoring (optional)	Deployment Readiness Score
Audit	Run Date

6.3 Project Directory Structure



7 Constraints

▲ Mandatory Constraints — Violations Result in Disqualification

The following constraints are non-negotiable. Submissions that violate any constraint will be disqualified regardless of output quality.

7.1 Technical

- TC1. Read-Only:** No write, update, or delete operations against RIS or any source data. The agent is a read-and-analyse pipeline only.
- TC2. No AI Exposure of Data:** Employee, grade, and skill data must not be transmitted to any external AI/LLM API or cloud service. R3 and R4 hybrid logic must execute locally.
- TC3. Configurable Grade-Band Mapping:** Junior / Mid / Senior bands must be loaded from `grade_band_map.csv`. No band assignments may be hardcoded in source code.
- TC4. Graceful Handling of Unmapped Grades:** If a grade value is not in the mapping file, exclude it from pyramid calculations and log a warning. Do not crash.
- TC5. Input Format Lock:** Accept the provided RIS file as-is. No manual pre-processing or column renaming by the evaluator.

7.2 Functional

- FC1. Advisory Only:** FDA recommends. It does not modify project structures or approve fresher allocations.
- FC2. Target Pyramid Immutability:** The 79 / 20 / 1 targets must not be changed, unless explicitly advised by RM contacts during KT.
- FC3. Schema Compliance:** Output column groups must match the specification. N/A for non-applicable fields, never blank.
- FC4. Frontend:** Frontend screen or web interface is required or permitted in this submission.

7.3 Known Limitations

Limitation	Impact
Grade-band mapping TBC	Pyramid calculations are only as accurate as the grade-band config. Must be confirmed at KT.
RIS skill data quality	Training and deployment suggestions depend on Technology and Skill Description fields being populated. Sparse data reduces recommendation quality.
No TalentX / HRMS in MVP	Fresher intake tracking and richer skill matching are limited to what RIS exposes.
Unit of analysis TBC	Project-level vs account-level aggregation must be confirmed at KT.
Active population filter TBC	Without a confirmed filter definition, active headcount figures may be inconsistent.

8 Submission Guidelines

1. Submit as: `TeamName_Agent03_FDA_Submission.zip`
2. ZIP must include: source code, `README.md`, the grade band mapping config file (with sample values), and both output reports from the provided test dataset.
3. Deadline: **[TODO: to be announced]**
4. Demo: Evaluator will supply a fresh RIS input file. Agent must run end-to-end without manual intervention.

RM Contact for Knowledge Transfer

For grade-band mapping confirmation, unit of analysis, active population filter definition, and skill taxonomy guidance, contact **Abdul, Phanindra** via the official hackathon channel.